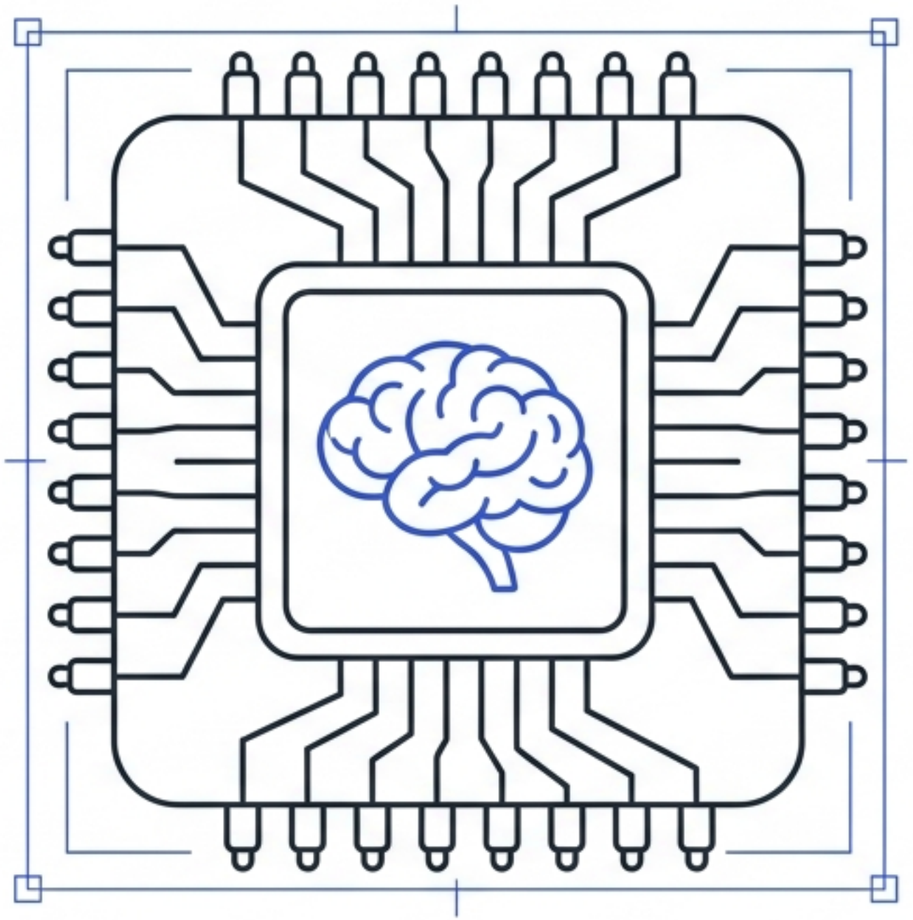
Microcontroller Fundamentals

IIT67-372 Internet of Things

Asst. Prof. Dr. Chanankorn Jandaeng

School of Informatics, Walailak University, Thailand

Meet the Brain of Smart Devices



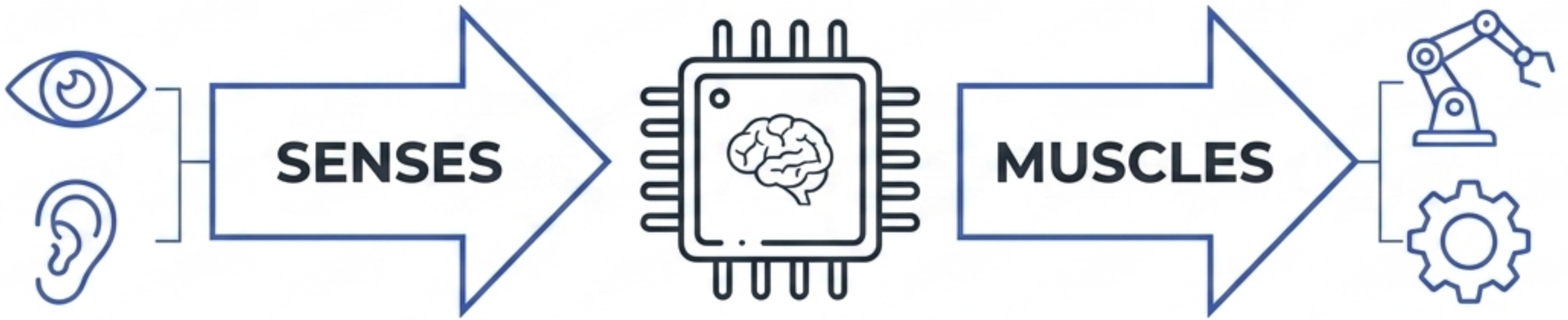
A **microcontroller** is a compact integrated system built onto a single tiny chip.

It combines a processor (the brain), memory, and ways to connect to the outside world.

They act as the hidden intelligence inside everyday embedded systems and IoT devices.

Real-life example: When your smart coffee maker knows exactly when to start brewing, a microcontroller is running the show.

GPIO: The Senses and Muscles



GPIO stands for General Purpose Input/Output.

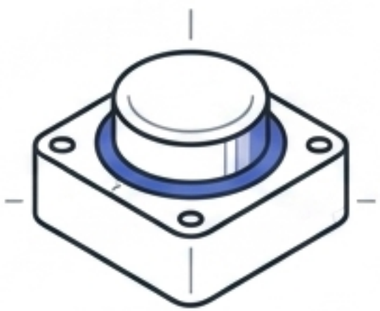
These are the physical metal pins that connect to external parts.

They allow the internal brain to interact directly with the physical world.

Real-life example:
Connecting a temperature sensor (a sense) or a spinning motor (a muscle).

Two-Way Communication: Input vs. Output

INPUT (Reading)



- The board reads signals coming in from external components.
- Example: Pressing a button on a TV remote.

OUTPUT (Sending)

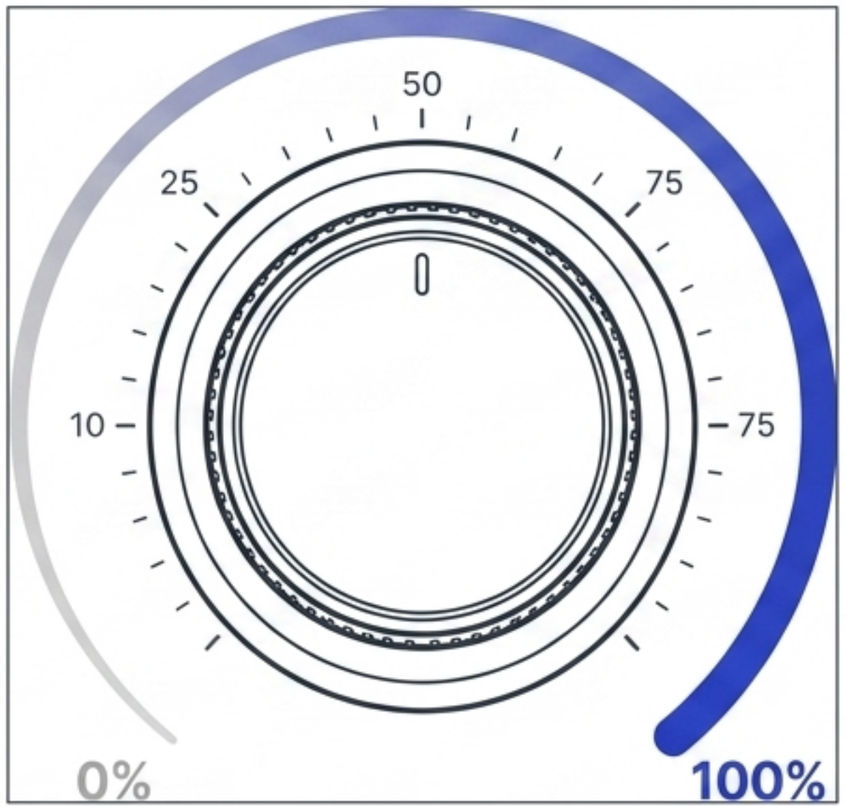
- The board sends signals out to control external components.
- Example: The television turning on in response.

Crucial Concept: Every GPIO pin can be specifically configured to act as either an input or an output.

Digital Signals are Black and White

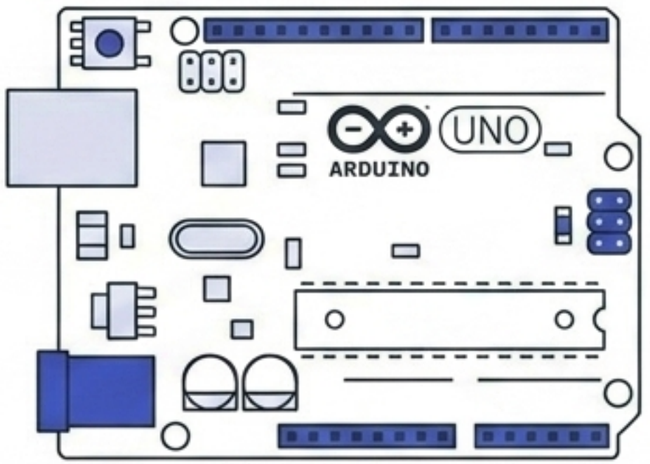
1. **Binary Communication:** Digital signals only understand two absolute states.
2. **Strict Values:** The values are strictly 0 or 1, which translates to LOW or HIGH electrical power.
3. **No In-Between:** It acts exactly like a simple on/off switch with no middle ground.
4. **Real-life example:** A standard room light is either fully turned on or completely turned off.

Analog Signals are Shades of Gray



- Analog signals represent continuous information with a wide, flowing range of voltage levels.
- Because computers only think in digital, they use an **ADC (Analog-to-Digital Converter)** to translate these ranges into numbers.
- This allows the board to understand precise, gradual changes in the environment rather than - than just black-and-white states.
- Real-life example: Turning a volume knob gradually from 0 to 100, or using a dimmer switch.

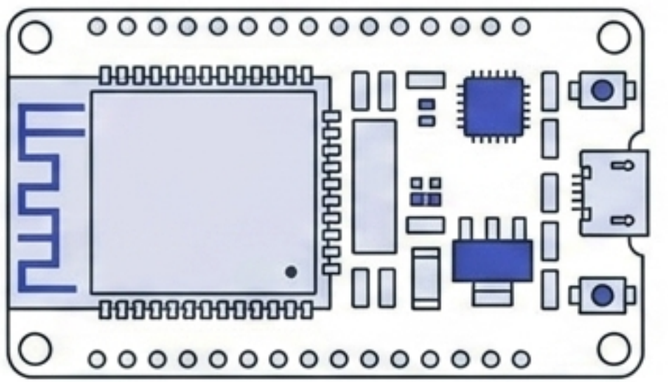
Two Famous Microcontrollers



Arduino Uno

The world's most popular foundational board for beginners learning basic electronics.

Project: Build a simple robot toy.



ESP32

A more advanced, powerful board designed specifically for the Internet of Things (IoT).

Project: Build a Wi-Fi-connected smart weather station.

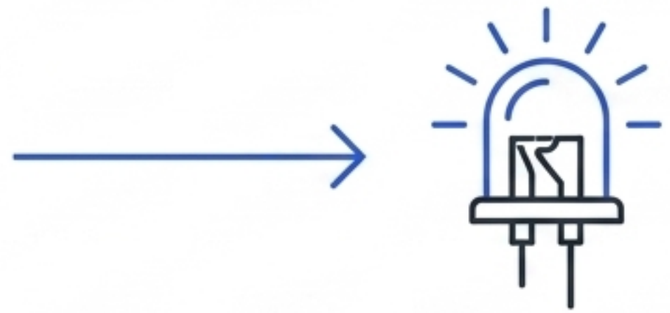
Both platforms are used globally by students and engineers to prototype new ideas.

Comparing the Hardware Platforms

Feature	Arduino Uno 	ESP32 
Digital Pins	14	**30+**
Analog Input	10-bit ADC (Basic)	**12-bit ADC (Precise)**
PWM Pins	Limited	Many
Built-in LED Pin	Pin 13	GPIO 2

Your First Project: The Blink Lab

Hello World



In software engineering, the very first lesson is always printing the words “Hello World” on a screen.

In embedded systems, the equivalent is making a single LED light blink on a physical board.

This simple task is a critical test. It verifies that your hardware and software are successfully interacting.

Real-life example: A blinking status light on your Wi-Fi router uses this exact foundational concept.

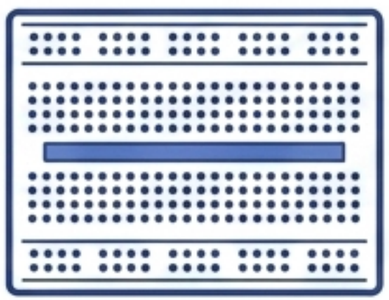
The Circuit: What You Need

LED

A tiny light bulb that glows when electricity passes through it.

220Ω Resistor

Acts like a traffic cop, slowing down the electricity so the LED doesn't burn out.



Breadboard

A plastic board with holes that lets you plug in wires without soldering metal.

Jumper Wires

The basic cables that carry the electrical signals between the components.

Wiring the Complete Loop

Electricity must always flow in a complete, unbroken loop to function properly.

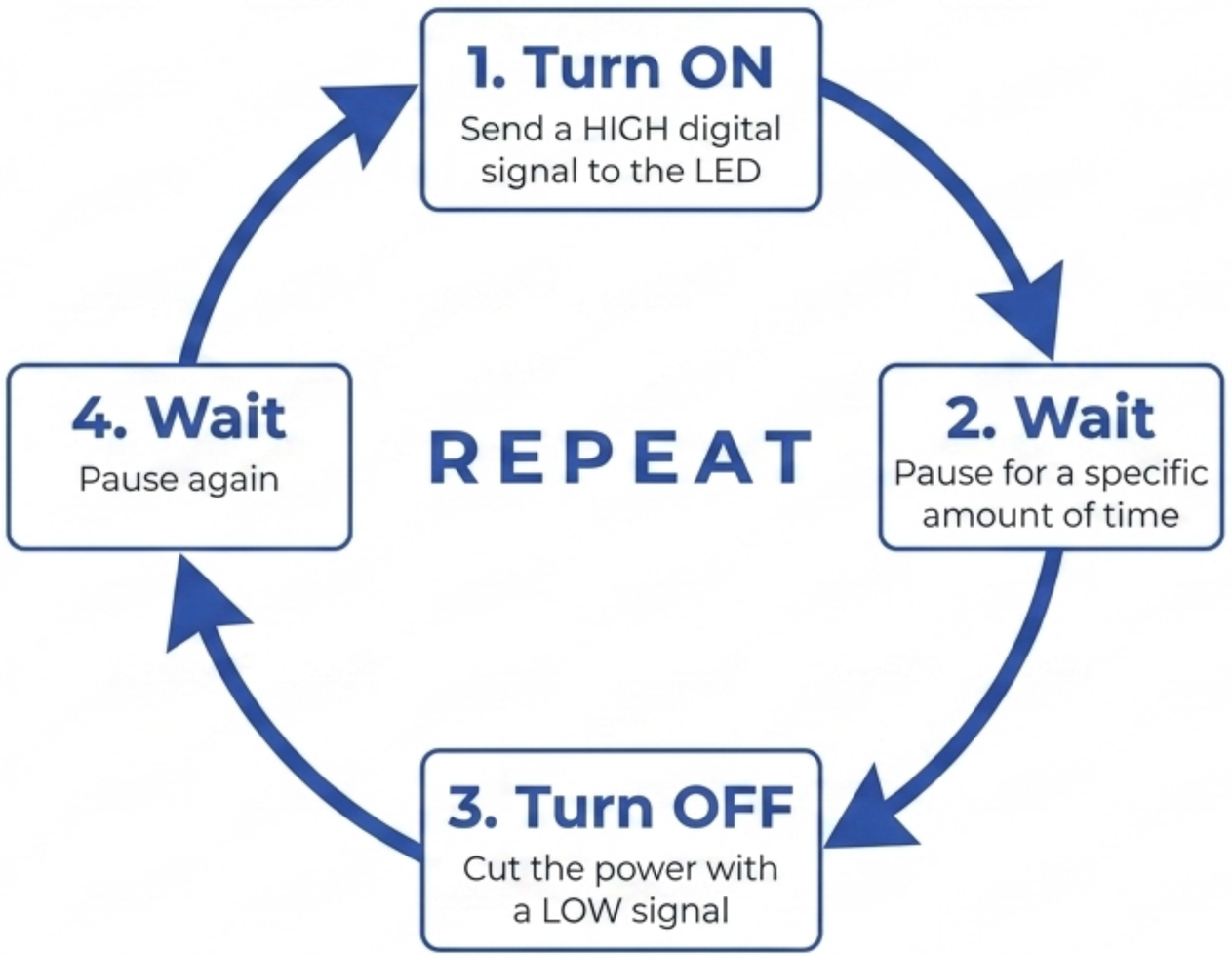
Step 1:
Connect the long leg of the LED (anode +) to the power source at Digital Pin 13.

Step 2:
Connect the short leg (cathode -) to the 220Ω Resistor.

Step 3:
Connect the other end of the Resistor to GND (Ground) to complete the loop.

The Logic Behind the Blinking

We write simple text instructions (code) to tell the hardware exactly what to do. Before looking at the syntax, we must understand the logical sequence.



Three Magic Coding Commands

```
pinMode(pin, OUTPUT)
```

Translation: Get ready, this specific pin is going to send signals out.

```
digitalWrite(pin, HIGH)
```

Translation: Turn the power ON (**HIGH**) right now. (Using **LOW** turns it OFF).

```
delay(1000)
```

Translation: Do nothing and hold this exact state for 1000 milliseconds (1 full second).

From Blinking Lights to Smart Homes

Basic Foundation

Blinking a tiny light uses the exact same embedded control logic as advanced robotics.

Smart Home

Replacing the LED with a heating element turns this project into a smart thermostat.

Industrial IoT

Replacing the LED with a heavy motor turns this project into an automated factory arm.

The Big Picture: Mastering this basic hardware-software interaction unlocks the foundation of all IoT technology.

Key Takeaways

- Microcontrollers act as the compact brains driving modern embedded systems and IoT devices.
- GPIO pins can be configured as Inputs (reading signals from sensors) or Outputs (sending signals to actions).
- Hardware communicates using basic Digital signals (0/1 switch) or complex Analog signals (continuous ranges).
- Writing basic code allows us to control physical electricity, proving successful hardware-software interaction.